

UNIVERSITÀ TELEMATICA INTERNAZIONALE UNINETTUNO

FACOLTÀ DI INGEGNERIA

Corso di Laurea Magistrale in
Ingegneria Informatica

ELABORATO FINALE

in

Progettazione del software

Sviluppo di una Pipeline di Big Data Analytics per la Classificazione in Tempo Reale di Aritmie Cardiache

CORRELATORE
Prof. Mazzei Mauro

CANDIDATO
Donadello Andrea

TODO Dedicar

Sommario

Si presenta un sistema per la classificazione *near-real-time* di flussi ECG (elettrocardiogramma) basato su un approccio **ibrido batch/streaming**.

Il problema è affrontato come una pipeline di decisione distribuita nella quale l'addestramento del modello avviene in modo *off-line* su dati storici, mentre l'inferenza è eseguita su eventi in arrivo con vincoli di latenza compatibili con scenari di monitoraggio continuo. L'architettura è stata implementata in **Scala** e combina un layer di addestramento batch, realizzato con **Apache Spark MLlib**, e un layer di inferenza in streaming, realizzato con **Spark Structured Streaming** che consuma messaggi da un broker **Kafka**.

Le *feature* utilizzate sono estratte a livello di battito dal dataset Kaggle *ECG Arrhythmia Classification Dataset* e riflettono contesto ritmico locale (intervalli **pre-RR** e **post-RR**), ampiezze delle onde P/T e del complesso QRS, durate degli intervalli PQ/QT/ST e descrittori morfologici del QRS su due derivazioni (lead II e V5). Questa rappresentazione riduce la dimensionalità del problema e consente di applicare modelli di *classical machine learning* su vettori strutturati, costruiti tramite **VectorAssembler**.

Il modello scelto per l'implementazione sperimentale è un **RandomForestClassifier** configurato con 100 alberi e profondità massima pari a 15, in modo da bilanciare capacità discriminativa, robustezza rispetto al rumore e semplicità di deployment.

La pipeline complessiva prevede quattro fasi principali:

1. Preparazione e pulizia dei dati storici per l'addestramento, comprensiva di normalizzazione delle feature e gestione degli outlier;
2. Serializzazione e deploy del modello su storage locale, così da rendere l'inferenza indipendente dalla fase di training;
3. Simulazione di ingestion in *real-time* tramite il componente **EcgPatientSimulator** che pubblica payload JSON sul topic **ecg-input-stream**;
4. Lettura dai topic Kafka, deserializzazione dinamica con **from_json**, applicazione del modello nel componente **EcgStreamingPredictor** e persistenza dei risultati in **Delta Lake** con *checkpointing* e **foreachBatch** per garantire tolleranza ai fault, recupero in caso di failure e comportamento *exactly-once* a livello di micro-batch.

La valutazione sperimentale adotta il `MulticlassClassificationEvaluator` per misurare l'accuratezza e analizzare il bilanciamento tra falsi positivi e falsi negativi, con particolare attenzione all'impatto clinico di quest'ultimi.

Accanto alla qualità predittiva, si discutono *latenze end-to-end*, *throughput* e comportamento della pipeline in presenza di componenti disaccoppiati, evidenziando il ruolo della messaggistica asincrona e dello storage transazionale.

Le evidenze sperimentali e i trade-off architetturali sono riportati nel dettaglio nei capitoli dedicati; possibili estensioni includono l'adozione di modelli deep (RNN/LSTM o Transformer) per analisi diretta del segnale raw, l'arricchimento del flusso con feature estratte su edge device e l'integrazione di dashboard analitiche con Delta Lake.

Indice

1	Introduzione	9
1.1	Contesto e motivazione	9
1.2	Approccio e scelte tecnologiche	9
1.3	Obiettivi	10
1.4	Contributi	10
1.5	Struttura della tesi	11
2	Dominio Applicativo e Dataset	12
2.1	Dominio applicativo: il segnale ECG	12
2.1.1	Elettrofisiologia e morfologia del battito	12
2.1.2	Variabilità della frequenza cardiaca	13
2.1.3	Rappresentazione computazionale e feature	13
2.1.4	Preprocessing e qualità del segnale	14
2.2	Dataset sperimentale: MIT-BIH tabellare (Kaggle)	15
2.2.1	Origine e caratteristiche del corpus PhysioNet	15
2.2.2	Schema del file CSV e volume dati	15
2.2.3	Schema di raggruppamento AAMI e etichette nel CSV	15
2.2.4	Sbilanciamento tra classi	17
2.2.5	Tipologie di anomalie contenute e limiti	17
2.3	Edge Computing e ruolo del simulatore	17

2.3.1	Funzione metodologica del simulatore	18
2.3.2	Scenario evolutivo: digital twin	18
2.4	Considerazioni etiche e implicazioni cliniche	18
3	Stack Tecnologico	20
3.1	Il linguaggio Scala: programmazione funzionale applicata ai Big Data	20
3.1.1	Immutabilità, composizione e type system	20
3.1.2	Confronto con alternative	21
3.2	Apache Spark: Core, Spark SQL e MLlib	21
3.2.1	Modello di esecuzione	21
3.2.2	Spark SQL, Catalyst e Tungsten	21
3.2.3	MLlib, Pipeline API e confronto con alternative	22
3.2.4	Structured Streaming: semantica temporale e trigger	22
3.3	Apache Kafka: architettura publish-subscribe e disaccoppiamento	23
3.3.1	Architettura log-centrica	23
3.3.2	Partizioni, consumer group e parallelismo	23
3.3.3	Modalità KRaft e deprecazione di ZooKeeper	24
3.3.4	Garanzie di consegna ed exactly-once	24
3.4	Delta Lake: il paradigma Data Lakehouse	24
3.4.1	Limiti di Parquet puro	24
3.4.2	Transaction log, ACID e schema	25
3.4.3	Time travel, VACUUM e audit clinico	25
3.4.4	Integrazione con Spark	25
3.5	Sintesi: requisiti e tecnologie adottate	25
4	Caso di studio	27
4.1	Pattern architetturali: Lambda, Kappa e posizionamento del progetto	27

4.2	Panoramica della pipeline	28
4.2.1	Flusso granulare dei dati	28
4.3	Vista fisica e deployment	28
4.3.1	Limiti del setup mono-nodo	29
4.4	Fase di addestramento: EcgModelTrainer	29
4.4.1	Flusso operativo	29
4.4.2	Scelta del classificatore e alternative	29
4.4.3	Output del trainer	30
4.5	Simulazione IoT: EcgPatientSimulator	30
4.5.1	Logica di produzione	30
4.5.2	Schema del payload e impatto sulla banda	30
4.5.3	Estensioni e limiti	30
4.6	Inferenza in tempo reale: EcgStreamingPredictor	31
4.6.1	Bootstrap dello schema JSON	31
4.6.2	Schema Registry e contratto dati	31
4.6.3	Pipeline di inferenza	31
4.6.4	Ciclo di vita del modello in produzione	32
4.6.5	Fault tolerance e exactly-once	32
4.7	Orchestrazione: AppRunner	32
4.8	Requisiti non funzionali	33
4.8.1	Disaccoppiamento	33
4.8.2	Idempotenza e consistenza	33
4.8.3	Scalabilità	33
4.8.4	Evolubilità dello schema	33
4.8.5	Observability	33
4.8.6	Sicurezza	33

Elenco delle figure

3.1	Dal codice Scala al piano eseguito: Catalyst e Tungsten nel percorso di una query Spark SQL.	22
-----	---	----

Elenco delle tabelle

2.1	Corrispondenza tra etichette nel CSV Kaggle e super-classi AAMI.	16
2.2	Raggruppamento AAMI delle annotazioni MIT-BIH (sintesi del contenuto clinico).	16
2.3	Feature per derivazione nel CSV MIT-BIH Arrhythmia Database.csv (prefisso 0_ o 1_).	16
3.1	Confronto tra Kafka con ZooKeeper e Kafka in modalità KRaft.	24
3.2	Mappatura requisiti-tecnologie.	25

Capitolo 1

Introduzione

1.1 Contesto e motivazione

Le patologie cardiovascolari rappresentano una delle prime cause di morbidità e mortalità a livello globale; il monitoraggio continuo dell'attività cardiaca tramite segnali ECG è pertanto centrale per la diagnosi precoce e la gestione clinica.

In molti scenari applicativi, l'interpretazione del segnale non può essere affidata esclusivamente a una valutazione umana *ex post*, poiché il valore informativo del battito risiede anche nella *tempestività* con cui viene rilevata un'anomalia. Da questo punto di vista, l'analisi automatica dei battiti cardiaci pone sfide tecniche sostanziali:

- La necessità di elaborazione **near-real-time**,
- La gestione di flussi ad alta velocità,
- La robustezza rispetto a fault e perdita di pacchetti,
- La capacità di mantenere *consistenza* tra fasi di ingestione, predizione e persistenza.

Nel dominio medico i trade-off tra **precisione** e **sensibilità** ricoprono un ruolo critico. I falsi negativi possono portare a omissioni diagnostiche rilevanti, mentre i falsi positivi possono introdurre rumore operativo e aumentare il carico di verifica manuale.

Ne deriva l'esigenza di una pipeline che non si limiti a produrre una classificazione accurata, ma che sia anche *interpretabile* dal punto di vista architetturale, scalabile nel numero di sorgenti e adatta a operare su infrastrutture distribuite. In tale prospettiva, il progetto affronta il problema della classificazione di battiti ECG come un caso di studio per l'integrazione tra **machine learning classico** e **data streaming**.

1.2 Approccio e scelte tecnologiche

In questo lavoro si affronta il problema della classificazione di battiti ECG in un contesto di streaming, privilegiando un'architettura che consenta **disaccoppiamento** tra ingestione e processing,

resilienza ai guasti e scalabilità orizzontale.

La soluzione proposta impiega tecnologie consolidate nello stack Big Data:

- **Scala**: implementazione del sistema e dei componenti della pipeline,
- **Apache Kafka**: broker Pub/Sub per la messaggistica asincrona,
- **Apache Spark**: elaborazione batch (MLlib) e streaming (Structured Streaming),
- **Delta Lake**: sink transazionale per la persistenza dei risultati.

Questa combinazione consente di separare in modo netto la responsabilità di acquisizione dei dati dalla responsabilità di inferenza, evitando che il carico di una fase condizioni in maniera diretta l'altra.

La scelta di una soluzione *ibrida batch/streaming* risponde a un'esigenza precisa: i modelli predittivi possono essere addestrati su archivi storici con costi computazionali più elevati, mentre l'inferenza deve operare con latenza contenuta e comportamento deterministico. Il progetto adotta quindi una pipeline che riproduce un ciclo realistico di produzione dei dati, addestramento e serving, offrendo un contesto coerente per valutare non soltanto la qualità del classificatore, ma anche la solidità dell'infrastruttura sottostante.

1.3 Obiettivi

Gli obiettivi perseguiti sono i seguenti:

1. Progettare e implementare una **pipeline end-to-end** per la classificazione di battiti ECG in near-real-time, rispettando requisiti di tolleranza ai fault, scalabilità e disaccoppiamento tra le componenti.
2. Sviluppare un processo di **addestramento batch** per un modello basato su Random Forest, includendo fasi di feature engineering, selezione delle variabili e validazione sperimentale.
3. Implementare un meccanismo di **ingestion realistico** tramite un simulatore (`EcgPatientSimulator`) che converte righe del CSV preprocessato (derivato dal MIT-BIH) in payload JSON e li pubblica su Kafka.
4. Eseguire **inferenza in streaming** con Spark Structured Streaming, garantendo atomicità delle scritture su Delta Lake tramite checkpointing e `foreachBatch`, così da preservare la continuità operativa in presenza di interruzioni.
5. Valutare la qualità del modello con metriche di classificazione e analizzare *latenze*, *throughput* e comportamento del sistema al variare del volume dei messaggi.

1.4 Contributi

Si elencano i contributi principali del lavoro:

- **Realizzazione di una pipeline sperimentale completa** in Scala che integra training batch e inferenza in streaming su dati ECG simulati, mantenendo la separazione tra livelli di acquisizione, elaborazione e persistenza.
- **Approccio pragmatico al feature engineering** su battiti cardiaci (intervalli pre-RR/post-RR, descrittori morfologici del QRS, durate PQ/QT/ST e coefficienti `qrs_morph` su due derivazioni) e definizione di uno schema JSON compatto per l'ingestion in Kafka.
- Dimostrazione dell'integrazione di Spark Structured Streaming con schema dinamico (`from_json`), checkpointing e scritture transazionali su Delta Lake per garantire resilienza, consistenza e ripetibilità delle elaborazioni.
- Analisi sperimentale completa della performance del modello (tramite `MulticlassClassificationEvaluator`) e delle proprietà non funzionali della pipeline, con attenzione a latenze, throughput e stabilità operativa.
- Formalizzazione di un caso di studio in cui una stessa architettura può essere riletta sia come sistema di prototipazione accademica sia come base per un futuro deployment in ambito e-Health.

1.5 Struttura della tesi

La trattazione è organizzata come segue:

- **Capitolo 1** descrive il dominio applicativo e il dataset utilizzato (MIT-BIH in versione tabellare da Kaggle), includendo considerazioni su feature cliniche, preprocessing del segnale e paradigma Edge Computing.
- **Capitolo 2** giustifica le scelte tecnologiche adottate, soffermandosi sui vantaggi di Scala, Spark, Kafka KRaft e Delta Lake nel contesto di pipeline distribuite.
- **Capitolo 3** illustra l'architettura complessiva e la scomposizione in tre moduli principali: `EcgModelTrainer` (batch), `EcgPatientSimulator` (ingestion) e `EcgStreamingPredictor` (streaming).
- **Capitolo 4** approfondisce dettagli implementativi, mostrando frammenti di codice e spiegando le scelte di configurazione della ML pipeline e del producer Kafka.
- **Capitolo 5** presenta la valutazione sperimentale, confrontando metriche di accuratezza, analisi di falsi positivi/negativi, e misure di latenze e throughput; infine, le conclusioni espongono valutazioni complessive e possibili linee di estensione verso modelli deep e integrazione analitica con Delta Lake.

I file sorgente principali del progetto contengono i componenti citati (`EcgModelTrainer`, `EcgPatientSimulator`, `EcgStreamingPredictor`) e la documentazione implementativa è richiamata nei capitoli successivi dove si analizzano implementazione e risultati. Questa impostazione consente di collegare in modo diretto la descrizione teorica con le scelte effettivamente adottate nel repository, evitando una trattazione puramente astratta.

Capitolo 2

Dominio Applicativo e Dataset

2.1 Dominio applicativo: il segnale ECG

Il segnale elettrocardiografico (ECG) rappresenta la proiezione elettrica dell'attività cardiaca misurata sulla superficie corporea. Dal punto di vista fisiologico, esso riflette la sequenza di depolarizzazione e ripolarizzazione delle camere cardiache e consente di osservare, con risoluzione temporale elevata, eventi che sono spesso immediatamente correlati allo stato elettrico del miocardio [15].

2.1.1 Elettrofisiologia e morfologia del battito

L'attivazione elettrica del cuore origina nel **nodo seno-atriale** (SA), che innesca la depolarizzazione degli atri. Il fronte di onda raggiunge poi i ventricoli attraverso il **nodo atrioventricolare** (AV), il **fascio di His** e il sistema delle **fibre di Purkinje**, che distribuiscono l'impulso sul miocardio ventricolare in modo coordinato. La sequenza temporale di queste fasi si traduce, sul tracciato, in segmenti e intervalli con significato clinico ben definito.

La morfologia tipica di un battito in ritmo sinusale è costituita dalle onde **P**, **QRS** e **T**: l'onda P corrisponde alla depolarizzazione atriale; il complesso QRS alla depolarizzazione ventricolare (con i deflessi Q, R e S che descrivono l'attivazione sequenziale del setto e delle pareti ventricolari); l'onda T alla ripolarizzazione ventricolare. Tra questi deflessi si misurano il **segmento PR** (conduzione atrioventricolare), il **segmento ST** (fase di ripolarizzazione precoce ventricolare) e l'**intervallo QT** (durata dell'attività ventricolare totale), spesso corretto per la frequenza cardiaca (QTc) nelle valutazioni cliniche.

La Figura ?? sintetizza, in forma schematica, la relazione tra fasi elettriche e morfologia osservabile. In applicazioni automatiche, la forma locale del QRS e la durata degli intervalli costituiscono descrittori ad alta densità informativa per la distinzione tra battiti normali e patologici [7].

Gli *intervalli RR* (distanza temporale tra picchi R consecutivi) sono la misura principale per la frequenza cardiaca e per la *Heart Rate Variability* (HRV). In presenza di aritmie, la distribuzione degli RR perde regolarità: ectopie, pause o tachicardie producono sequenze che un classificatore può sfruttare insieme alla morfologia del battito isolato [7], [25].

2.1.2 Variabilità della frequenza cardiaca

La HRV quantifica le fluttuazioni dell'intervallo RR attorno al valore medio e riflette, in parte, l'equilibrio tra i sistemi simpatico e parasimpatico. Le linee guida del Task Force del 1996 [25] distinguono due famiglie di misure, entrambe rilevanti per il feature engineering adottato in questo lavoro.

Nel **dominio del tempo**, le metriche più diffuse includono:

- **SDNN**: deviazione standard degli intervalli NN (battiti di origine sinusale);
- **RMSSD**: radice quadrata della media dei quadrati delle differenze successive tra intervalli NN;
- **pNN50**: percentuale di coppie di intervalli NN consecutivi la cui differenza supera 50 ms.

RMSSD e pNN50 sono particolarmente sensibili alle componenti ad alta frequenza, associate all'attività vagale a breve termine.

Nel **dominio della frequenza**, lo spettro della serie RR (tipicamente ricampionata e de-trendata) viene suddiviso in bande: la componente **LF** (circa 0.04–0.15 Hz) e la componente **HF** (circa 0.15–0.40 Hz), quest'ultima legata alla respirazione e al tono parasimpatico. Il **rapporto LF/HF** è usato in letteratura come indicatore sintetico dell'equilibrio autonomico, sebbene la sua interpretazione clinica richieda cautela e contesto del paziente [25].

Nel dataset impiegato, metriche HRV globali (SDNN, RMSSD, spettro LF/HF) non sono calcolate in tempo reale dalla pipeline Scala; il CSV contiene però intervalli RR locali (**pre-RR**, **post-RR**) e descrittori morfologici già estratti offline dal segnale grezzo. Il classificatore opera quindi su un livello di astrazione che unifica morfologia istantanea e contesto ritmico immediato attorno al battito corrente.

2.1.3 Rappresentazione computazionale e feature

Dal punto di vista computazionale, il battito ECG può essere rappresentato come una **sequenza multiscala**: una componente temporale (distanza tra eventi) e una componente morfologica (forma del tracciato locale). Questa doppia prospettiva consente di trasformare un segnale continuo in un insieme di osservazioni etichettate, ciascuna associata a un istante clinicamente significativo, adatto a modelli supervisionati che richiedono vettori numerici stabili nel tempo.

Nel file MIT-BIH `Arrhythmia Database.csv` del corpus Kaggle [22], [23], ogni riga descrive un singolo battito con **32 feature numeriche** (16 per ciascuna delle due derivazioni, prefissi `0_` per lead II e `1_` per lead V5), oltre a **record** (identificativo registrazione) e **type** (etichetta):

- **Contesto ritmico**: **pre-RR** e **post-RR** (intervalli R–R rispettivamente precedente e successivo al battito corrente, in ms) [7].
- **Morfologia delle onde**: ampiezze **pPeak**, **qPeak**, **rPeak**, **sPeak**, **tPeak**.
- **Durata degli intervalli**: **pq_interval**, **qrs_interval**, **st_interval**, **qt_interval**.

- **Forma del QRS:** cinque coefficienti `qrs_morph0`–`qrs_morph4` che riassumono la morfologia del complesso [8].

Tali attributi costituiscono il *vettore di ingresso* per modelli di machine learning classico (Random Forest nel caso di studio) e sono prodotti da una catena di preprocessing che precede l’addestramento batch. La scelta di non trasmettere il segnale raw riduce la complessità del modello, limita il volume dei messaggi Kafka e rende esplicita la semantica di ogni campo nel payload JSON.

2.1.4 Preprocessing e qualità del segnale

La qualità del segnale condiziona in modo diretto l’affidabilità delle feature. Prima della segmentazione beat-to-beat, la letteratura propone una sequenza di operazioni che, nel nostro flusso, sono applicate offline sulla fase di costruzione del CSV, ma che in un deployment edge replicherebbero lo stesso ordine logico [8], [21].

Filtraggio. Un **filtro passa-alto** (tipicamente con frequenza di taglio intorno a 0.5 Hz) attenua il *baseline wander*, derivante da respirazione, movimento del paziente o deriva degli elettrodi. Un **filtro notch** centrato a 50 Hz o 60 Hz rimuove l’interferenza della rete elettrica (*power-line interference*). Un **filtro passa-basso** (spesso tra 35 e 40 Hz, coerente con la banda utile del MIT–BIH) limita il rumore muscolare e le componenti ad alta frequenza poco informative per la classificazione dei battiti. In pratica, la combinazione di questi stadi equivale a un filtraggio passa-banda nell’intervallo 0.5–40 Hz citato nelle specifiche del database.

Rilevamento dei picchi R. L’algoritmo di *Pan–Tompkins* [21] è un riferimento consolidato per la detection in tempo quasi reale. La pipeline prevede: (i) filtraggio passa-banda; (ii) derivata per enfatizzare le pendenze del QRS; (iii) elevamento al quadrato per uniformare la polarità; (iv) integrazione a finestra mobile per smussare il segnale; (v) soglia adattiva su due livelli per distinguere picchi veri da artefatti. La robustezza dipende dalla calibrazione delle soglie e dalla qualità del tracciato: in presenza di aritmie con morfologie atipiche, un detector generico può richiedere post-processing o regole cliniche aggiuntive.

Indice di qualità del segnale. Per evitare che artefatti di movimento o contatto degradino le feature, si adottano *Signal Quality Index* (SQI) che combinano metriche nel dominio del tempo, della frequenza o della forma d’onda [14]. Battiti associati a tratti di segnale con SQI insufficiente possono essere scartati, marcati come *unknown* o inviati su un canale di quarantena, riducendo la propagazione di errori verso il classificatore. In una pipeline distribuita, questa valutazione ha anche valenza architetturale: filtrare a monte significa trasmettere verso il broker solo eventi affidabili e alleggerire il carico del livello streaming.

Il preprocessing può dunque essere letto come **normalizzazione semantica**: il segnale fisiologico diventa una rappresentazione tabellare allineata al modello ML. In produzione, la fase può risiedere sul dispositivo wearable o su un gateway, così da inviare al data center feature già validate anziché campioni raw ad alta frequenza.

2.2 Dataset sperimentale: MIT–BIH tabellare (Kaggle)

Per la fase sperimentale si utilizza la versione tabellare del **MIT–BIH Arrhythmia Database** pubblicata nel corpus Kaggle *ECG Arrhythmia Classification Dataset* [22], [23], derivato dalla raccolta originale su PhysioNet [10], [18]. Il bundle Kaggle contiene più file CSV (MIT–BIH, MIT–BIH Supraventricular, INCART, SCD Holter); il prototipo Scala legge tramite `DATASET_PATH` i file `*.csv` presenti nella directory configurata: nelle sperimentazioni descritte in questa tesi si assume il file `MIT–BIH Arrhythmia Database.csv`, così da restare allineati al benchmark MIT–BIH senza mescolare registrazioni di altri database.

2.2.1 Origine e caratteristiche del corpus PhysioNet

Il MIT–BIH originale [10], [18] è una raccolta acquisita tra il 1975 e il 1979 presso il Beth Israel Hospital (oggi Beth Israel Deaconess Medical Center), con campionamento a 360 Hz e risoluzione di 11 bit su circa 10 mV di dinamica. Il corpus è diventato, nel tempo, uno *benchmark* de facto per la classificazione automatica dei battiti.

Le proprietà salienti del database grezzo sono le seguenti:

- 48 registrazioni di circa 30 minuti ciascuna, da 47 soggetti (due tracce per un paziente).
- Due canali per registrazione, derivazioni differenti (tipicamente lead I e una derivazione del lead II modificato).
- Annotazioni beat-to-beat curate da più esperti, con codici simbolici per oltre venti categorie di battiti ed eventi non ritmici.

La versione CSV impiegata nel progetto non contiene campioni raw del segnale, bensì battiti già segmentati con feature estratte offline (filtraggio, rilevamento QRS, calcolo di intervalli e descrittori morfologici) [7], [23]: la pipeline Scala legge questi archivi senza ripetere sul cluster l'intera catena descritta in §2.1.4.

2.2.2 Schema del file CSV e volume dati

Il file `MIT–BIH Arrhythmia Database.csv` contiene **100 689** battiti da **44** registrazioni (colonna `record`), con **34** colonne totali (identificativo, etichetta e 32 feature). Quattro tracce presenti nel MIT–BIH grezzo non compaiono nel CSV preprocessato, probabilmente escluse in fase di estrazione per qualità del segnale o annotazioni incomplete. Ogni riga corrisponde a un battito; le feature duplicano la stessa semantica su due derivazioni (lead II, prefisso `0_`; lead V5, prefisso `1_`), come riassunto in Tabella 2.3.

2.2.3 Schema di raggruppamento AAMI e etichette nel CSV

Per confrontare i risultati con la letteratura clinica e con lo standard AAMI EC57:2012 [5], le annotazioni originali del MIT–BIH vengono raggruppate in cinque **super-classi**, come proposto

da de Chazal et al. [7] e adottato nel Capitolo ???. Nel CSV Kaggle, la colonna **type** adotta una nomenclatura leggermente diversa dalle sigle AAMI letterali, ma con corrispondenza biunivoca (Tabella 2.1).

Tabella 2.1: Corrispondenza tra etichette nel CSV Kaggle e super-classi AAMI.

CSV (type)	AAMI	Significato
N	N	Normale
SVEB	S	Ectopia supraventricolare
VEB	V	Ectopia ventricolare
F	F	Fusione
Q	Q	Non classificabile

Tabella 2.2: Raggruppamento AAMI delle annotazioni MIT–BIH (sintesi del contenuto clinico).

Classe	Sigla	Contenuto tipico
N	Normale	Battiti sinusali, blocchi di branca sinistro (L), blocchi di branca destro (R)
S	Supraventricolare	Ectopie atriali (A), aberranti (a), nodali (J), ritmi supraventricolari
V	Ventricolare	Contrazioni premature ventricolari (V), ectopie ventricolari (E)
F	Fusion	Battiti di fusione (F)
Q	Unknown	Non classificabili, rumore marcato (/), altre etichette non mappate

La Tabella 2.2 non esaurisce ogni codice del database, ma fissa il vocabolario con cui si discutono metriche e errori clinici nel resto della tesi. Lo **StringIndexer** di Spark ML indicizza i valori testuali effettivi (N, SVEB, VEB, F, Q) senza rinomina esplicita verso le sigle AAMI: la mappatura resta a carico dell’analisi dei risultati.

Tabella 2.3: Feature per derivazione nel CSV MIT-BIH *Arrhythmia Database.csv* (prefisso 0_ o 1_).

Suffisso colonna	Contenuto
pre-RR, post-RR	Intervalli R–R precedente e successivo (ms)
pPeak ... tPeak	Ampiezze onde P, Q, R, S, T
pq_interval, qrs_interval, st_interval, qt_interval	Durate intervalli/segm. (ms)
qrs_morph0–qrs_morph4	Descrittori morfologici del complesso QRS

2.2.4 Sbilanciamento tra classi

La distribuzione dei battiti nel file `MIT-BIH Arrhythmia Database.csv` è fortemente *sbilanciata* (conteggi assoluti sul corpus completo): N 90 083 (89,5%), VEB 7 009 (7,0%), SVEB 2 779 (2,8%), F 803 (0,8%), Q 15 (<0,1%). Tale proporzione riflette la prevalenza clinica dei ritmi normali in registrazioni ambulatoriali selezionate, ma penalizza i modelli che ottimizzano l'accuratezza globale senza pesi per classe.

Durante l'addestramento si possono considerare **class weighting**, undersampling della classe maggioritaria o oversampling sintetico delle minoritarie; ciascuna strategia introduce trade-off (perdita di informazione, rischio di overfitting sulle classi rare, tempi di training più lunghi). Nel prototipo attuale non è implementato alcun ribilanciamento esplicito: la scelta è coerente con l'obiettivo di dimostrare l'architettura end-to-end, ma va tenuta presente nell'interpretazione delle metriche del Capitolo ??.

Dal punto di vista sperimentale, il MIT-BIH offre un compromesso favorevole tra complessità e riproducibilità: registrazioni sufficientemente eterogenee per stressare la pipeline, ma di dimensione gestibile in un contesto accademico in cui l'obiettivo principale non è il record di performance isolato, bensì la validazione di un flusso integrato dal dato storico allo streaming.

2.2.5 Tipologie di anomalie contenute e limiti

Le annotazioni includono ectopie ventricolari (PVC), alterazioni di conduzione, battiti supraventricolari e, in alcune tracce, eventi rari o tratti rumorosi che mettono alla prova la segmentazione. Il database non copre tutte le condizioni cliniche: aritmie prolungate come la fibrillazione atriale persistente o pattern legati a insufficienza cardiaca avanzata possono essere sottorappresentati.

La *generalizzabilità* verso altre popolazioni (età, sesso, patologia, dispositivo di acquisizione) non va assunta per default. I pazienti del MIT-BIH sono in gran parte adulti ricoverati o in follow-up cardiologico negli Stati Uniti della fine degli anni Settanta; un modello addestrato solo su questa distribuzione può degradare su cohort europee, pediatriche o su segnali da wearable consumer con banda e rumore differenti.

Per un classificatore supervisionato, la coerenza tra annotazioni, segmentazione e feature è tanto importante quanto l'algoritmo di apprendimento: errori a monte si propagano come etichette rumorose o descrittori distorti, limitando il soffitto di performance raggiungibile anche con modelli più complessi.

2.3 Edge Computing e ruolo del simulatore

Nel monitoraggio cardiaco continuo, la posizione dell'elaborazione lungo il percorso dato-cloud non è banale. Si distinguono almeno tre livelli [11]:

- **Cloud:** capacità computazionale elevata, storage centralizzato, latenza di rete non trascurabile;
- **Fog:** nodi intermedi (ospedale, edge data center) che aggregano flussi da più dispositivi;

- **Edge:** elaborazione sul wearable, sul Holter o sul gateway domestico, a ridosso del sensore.

Per l’ECG, spostare filtraggio, QRS detection e estrazione delle feature verso l’edge risponde a tre esigenze concrete: ridurre la latenza tra evento cardiaco e allarme; diminuire il traffico (un battito con 32 feature numeriche occupa ordini di grandezza in meno di un secondo di segnale a 360 Hz); limitare l’esposizione di dati biometrici raw oltre il perimetro controllato del paziente.

Dispositivi come Holter multicanale, patch ECG monouso e braccialetti con singola derivazione implementano già, in parte, questa logica: acquisiscono a alta frequenza localmente, poi trasmettono riassunti, eventi o tratti selezionati. Il progetto di tesi non dispone di hardware dedicato; il componente `EcgPatientSimulator` emula il comportamento di un producer edge che pubblica feature già calcolate sul topic Kafka `ecg-input-stream`, in formato JSON.

2.3.1 Funzione metodologica del simulatore

Il simulatore consente di esercitare la pipeline end-to-end in laboratorio: stesso schema colonne del CSV di training, stesso topic, stesso consumer Spark, senza dipendenze da firmware o protocolli proprietari. In uno scenario operativo, lo sostituirebbe un modulo on-device (o un gateway) con `KafkaProducer` o equivalente MQTT verso il broker.

L’approccio edge comporta vincoli noti: CPU e memoria limitate, consumo energetico, necessità di aggiornare modelli e soglie senza interrompere l’acquisizione. Per la tesi, questi aspetti restano fuori dall’implementazione, ma orientano le scelte architetturali del Capitolo 4 (payload compatto, disaccoppiamento tramite Kafka, training batch separato dall’inferenza streaming).

La riproducibilità degli esperimenti ne beneficia: la sequenza di messaggi può essere rigenerata con la stessa logica (`limit(1000)`, intervallo di un secondo), permettendo confronti tra configurazioni del predictor o del checkpoint senza variabilità legata a dispositivi reali.

2.3.2 Scenario evolutivo: digital twin

A orizzonte più ampio, i flussi ECG da molteplici fonti potrebbero alimentare un *digital twin* cardiaco del paziente: modello che integra storico clinico, eventi in tempo quasi reale e predizioni per simulare risposte a terapie o per prioritizzare alert. Il prototipo descritto in questa tesi costituisce un mattone elementare di tale visione—ingestion, classificazione, persistenza versionata—senza pretendere di realizzarne la complessità clinica e regolatoria.

2.4 Considerazioni etiche e implicazioni cliniche

Nell’applicazione di modelli ML a dati clinici, l’impatto di **falsi negativi** e **falsi positivi** va valutato prima delle metriche aggregate. Un falso negativo su una ectopia ventricolare ripetuta può ritardare un intervento; un falso positivo aumenta il carico di revisione per il personale e, nel lungo periodo, riduce la fiducia nel sistema di alert.

Il problema è dunque operativo oltre che statistico: output, latenza e priorità degli allarmi devono essere allineati al contesto (telemetria domiciliare, reparto, terapia intensiva), dove i trade-off tra sensibilità e specificità non coincidono.

Quadro regolatorio. In Europa, software destinato a finalità mediche può rientrare nel campo del Regolamento (UE) 2017/745 sui dispositivi medici (MDR) [9]. I sistemi di supporto alla diagnosi basati su algoritmi—spesso indicati come *Software as a Medical Device* (SaMD)—richiedono gestione del rischio, tracciabilità, validazione clinica e monitoraggio post-market. Il prototipo di questa tesi non è certificato né destinato all’uso clinico; le considerazioni regolatorie servono a inquadrare il divario tra dimostrazione accademica e prodotto deployabile.

Validazione, interpretabilità e bias. Un’adozione responsabile presuppone validazione su popolazioni e dispositivi diversi da quelli del training, documentazione delle prestazioni per sottogruppo e strumenti di interpretabilità (importanza delle feature, analisi degli errori per classe AAMI). Il MIT–BIH, per quanto prezioso, introduce *bias* demografico e tecnologico: campione storico, non necessariamente rappresentativo dell’utenza attuale né delle acquisizioni da wearable.

Infine, la pipeline deve essere leggibile da chi risponde dei dati: finalità del trattamento, minimizzazione (feature anziché raw dove possibile), retention su Delta Lake e accessi al broker vanno progettati con pari attenzione rispetto all’accuratezza del classificatore. Senza questi accorgimenti, anche un modello accurato resterebbe confinato al laboratorio.

Capitolo 3

Lo Stack Tecnologico

3.1 Il linguaggio Scala: programmazione funzionale applicata ai Big Data

Scala è un linguaggio multi-paradigma che combina programmazione orientata agli oggetti e programmazione funzionale sulla Java Virtual Machine (JVM). La sua progettazione punta a un *type system* espressivo, a immutabilità per default nelle strutture dati e a interoperabilità diretta con l'ecosistema Java, caratteristiche rilevanti per sistemi distribuiti di medie e grandi dimensioni [20].

Nel contesto di questo lavoro, Scala non è stato scelto per preferenze sintattiche, bensì perché rappresenta il *linguaggio nativo* di Apache Spark. Le API di Spark sono scritte in Scala; le trasformazioni su `Dataset` tipizzati sfruttano *encoder* derivati dal compilatore, evitando la serializzazione Python-JVM di PySpark, che introduce overhead non trascurabile quando il volume di eventi cresce [27].

Il progetto adotta la versione **2.13.18** (`build.sbt`), con prefisso di package `it.utiu.thesis`, in linea con Spark 4.1.1 e con le ottimizzazioni delle collezioni della serie 2.13.

3.1.1 Immutabilità, composizione e type system

Un principio ricorrente nel codice è l'uso di trasformazioni *lazy* su `DataFrame` anziché mutazioni esplicite. In `EcgModelTrainer`, la catena `read` $\rightarrow$ `na.drop` $\rightarrow$ `randomSplit` costruisce nuove viste logiche senza modificare il dataset precedente; la pipeline ML concatena `StringIndexer`, `VectorAssembler` e `RandomForestClassifier` in un unico `Pipeline` immutabile.

Le **case class** e il **pattern matching** facilitano la modellazione di strutture dati e la gestione di casi limite (ad esempio, assenza del modello su disco in `AppRunner`) con esaustività verificata dal compilatore. Nel dominio Spark, il vantaggio del type system emerge soprattutto con `Dataset[T]`: gli *implicit encoder* permettono serializzazione efficiente di tipi custom, a differenza di PySpark dove gran parte del percorso resta reflection-based e più costoso su piccoli batch ad alta frequenza.

In un sistema con thread concorrenti (predictor in background, simulatore sul main thread), l'assenza di stato condiviso mutabile tra componenti riduce race condition e semplifica il ragionamento sul comportamento in failure.

3.1.2 Confronto con alternative

PySpark offre una curva di apprendimento più rapida, ma ogni invocazione attraversa Py4J. Con un messaggio al secondo il costo è trascurabile; con centinaia di battiti al minuto per paziente e più stream paralleli, la latenza di bridging diventa misurabile. Java puro manca della concisione per esprimere pipeline dichiarative in poche righe. Scala resta, per questo progetto, il compromesso preferito: runtime JVM, API Spark native, codice batch e streaming nello stesso repository.

3.2 Apache Spark: Core, Spark SQL e MLlib

Apache Spark generalizza MapReduce con operazioni in-memory su strutture resilienti (RDD e, oggi, tabelle SQL) [26], [27]. Il progetto utilizza **Spark 4.1.1** con moduli `spark-core`, `spark-sql`, `spark-mllib` e connettore `spark-sql-kafka-0-10`.

3.2.1 Modello di esecuzione

Spark organizza il lavoro attorno a un *driver* e a uno o più *executor*. Il driver costruisce un DAG di *stage*; ogni stage contiene task parallelizzabili su partizioni di dati. In `Config.scala`, `SPARK_MASTER = "local[*]"` impiega tutti i core della macchina host su un singolo nodo: adeguato alla prototipazione, non rappresentativo di un cluster YARN, Kubernetes o Databricks.

3.2.2 Spark SQL, Catalyst e Tungsten

Spark SQL espone un'API dichiarativa su tabelle strutturate. L'ottimizzatore **Catalyst** [4] trasforma le operazioni attraverso una catena di fasi ben definita:

1. **Parsing**: analisi sintattica della query o del piano DataFrame;
2. **Analysis**: risoluzione di nomi, tipi e vincoli (catalogo);
3. **Logical optimization**: regole di rewrite (pushdown di filtri e proiezioni, pruning di colonne);
4. **Physical planning**: scelta di algoritmi (broadcast join, shuffle hash join, ecc.);
5. **Preparation**: generazione di codice eseguibile.

Nel trainer, operazioni come `groupBy("type").count()` dopo la lettura CSV beneficiano di predicate e projection pushdown senza intervento manuale. Nel predictor, il parsing `from_json`

e la **transform** del modello ML si integrano nello stesso planner, evitando materializzazioni intermedie superflue quando il motore può fonderle.

Tungsten [19] completa Catalyst con esecuzione orientata alla CPU e alla memoria: gestione off-heap, formato binario columnar interno e *whole-stage code generation*, che compila catene di operatori in un unico bytecode JVM. Su micro-batch ripetitivi (filtro, parsing JSON, proiezione colonne output), il codegen riduce il overhead per record rispetto a un interprete riga per riga.

La Figura 3.1 sintetizza il percorso dal codice applicativo all'esecuzione.

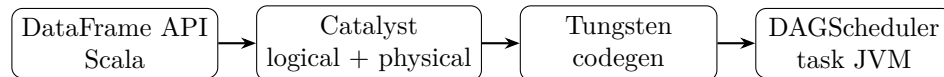


Figura 3.1: Dal codice Scala al piano eseguito: Catalyst e Tungsten nel percorso di una query Spark SQL.

3.2.3 MLlib, Pipeline API e confronto con alternative

MLlib espone algoritmi distribuiti tramite **Estimator**, **Transformer** e **Pipeline** [17]. Salvare `pipelineModel.write().save(...)` persiste in un colpo solo indicizzatore, assemblatore e Random Forest; `PipelineModel.load` garantisce che l'inferenza streaming replichi esattamente le trasformazioni del training.

Rispetto a **scikit-learn**, MLlib scala su cluster e condivide runtime con Structured Streaming, ma offre un catalogo algoritmi più ristretto e meno comodo per esperimenti rapidi su singola macchina. Rispetto a **TensorFlow/PyTorch**, MLlib non punta al deep learning end-to-end: reti neurali richiederebbero export ONNX, serving dedicato o integrazioni non native alla Pipeline API, aumentando il *technical debt* operativo descritto in letteratura per sistemi ML ibridi [24].

XGBoost4J-Spark e **GBTCClassifier** (MLlib) sono alternative plausibili per tabular data; Random Forest è stato preferito per integrazione nativa nella Pipeline, assenza di dipendenze aggiuntive e buona gestione di feature eterogenee senza scaling obbligatorio. La limitazione principale resta l'assenza di tuning sistematico (**CrossValidator**) nel prototipo, delegato a estensioni future.

3.2.4 Structured Streaming: semantica temporale e trigger

Structured Streaming tratta il flusso come tabella illimitata aggiornata nel tempo [2]. L'API replica il batch: `readStream`, trasformazioni, `writeStream`.

Processing time ed event time. Per default, il motore ragiona in **processing time** (arrivo dei record al consumer). Con **event time** e **watermark**, Spark gestisce record in ritardo fino a una soglia configurabile; oltre quella soglia, gli eventi tardivi possono essere scartati o inviati a un side output. Nel caso ECG del simulatore, timestamp e ritmo di invio sono allineati: non è stato necessario configurare watermark, ma un Holter reale con buffer offline richiederebbe event time esplicito sul campo temporale del payload.

Trigger. I trigger principali sono: `ProcessingTime(interval)` (micro-batch periodici), `Once / AvailableNow` (elaborazione fino a fine log, utile per catch-up), e il default *as fast as possible* adottato dal predictor, che avvia un micro-batch non appena sono disponibili dati da Kafka.

Micro-batch vs continuous processing. Structured Streaming implementa, di fatto, micro-batch su Spark SQL; la modalità *continuous processing* (latenza sub-secondo) resta sperimentale e limitata nel set di operatori. Per la tesi, il modello a micro-batch è accettabile: latenza end-to-end nell'ordine di centinaia di millisecondi resta compatibile con monitoraggio *near-real-time*, come discusso nel Capitolo ??.

Nel componente `EcgStreamingPredictor`, Structured Streaming consuma Kafka, applica `pipelineModel.transform` e scrive su Delta tramite `foreachBatch`. Questa scelta consente logica custom (`persist`, `isEmpty`, logging) a fronte di una gestione più esplicita del sink rispetto a `format("delta")` nativo.

3.3 Apache Kafka: architettura publish-subscribe e disaccoppiamento

Apache Kafka organizza la messaggistica attorno a *topic*, *partizioni* e *offset* [11], [13]. I producer appendono record immutabili; i consumer leggono tramite *consumer group* con bilanciamento del carico.

3.3.1 Architettura log-centrica

Ogni topic è un log replicato e partizionato, non una coda che consuma e dimentica. Più consumer possono rileggere lo stesso flusso a offset diversi (inferenza real-time, job di riprocessamento batch, audit).

Nel progetto, `ecg-input-stream` bufferizza tra `EcgPatientSimulator` e `EcgStreamingPredictor`: il producer può inviare un battito al secondo mentre Spark elabora micro-batch con latenza variabile, senza vincolare le due velocità.

3.3.2 Partizioni, consumer group e parallelismo

Una **partizione** è l'unità di ordinamento e parallelismo: i record con la stessa chiave finiscono nella stessa partizione, preservando l'ordine per chiave. Il numero massimo di consumer attivi in un gruppo che leggono in parallelo un topic è, in prima approssimazione, il numero di partizioni del topic. Con una sola partizione (default in sviluppo), il throughput di consumo Spark resta limitato a un task per micro-batch; aumentare le partizioni e `spark.default.parallelism` è il primo passo verso scaling orizzontale.

Structured Streaming gestisce internamente gli offset Kafka nel checkpoint: al restart, riprende dall'ultimo offset committato insieme al batch Delta, non dal solo puntatore del consumer Java.

3.3.3 Modalità KRaft e deprecazione di ZooKeeper

Storicamente, Kafka usava ZooKeeper per metadati di cluster. KRaft (Kafka Raft) introduce un quorum interno [1], riducendo componenti da operare.

Il `docker-compose.yml` del repository configura un nodo combinato broker+controller, quorum `1@localhost:9093`, `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1`. In produzione, broker e controller andrebbero separati con replication factor ≥ 3 .

Tabella 3.1: Confronto tra Kafka con ZooKeeper e Kafka in modalità KRaft.

Aspetto	ZooKeeper	KRaft
Componenti esterni	Sì (cluster ZK separato)	No
Tempo di failover	Secondi-decine di s	Sub-secondo (target)
Complessità operativa	Alta	Ridotta
Scalabilità metadata	Limitata	Migliorata

3.3.4 Garanzie di consegna ed exactly-once

Kafka supporta:

- **at-most-once**: messaggi possono perdersi, nessun duplicato;
- **at-least-once**: nessuna perdita, possibili duplicati al retry;
- **exactly-once** (EOS): producer idempotente + transazioni quando producer e consumer partecipano allo stesso cluster transazionale.

Spark Structured Streaming, con checkpoint e sink idempotente, punta a **exactly-once a livello di micro-batch**: ogni batch è processato una volta anche dopo restart del driver, a patto che la scrittura su Delta e il commit degli offset avvengano nella stessa transazione logica del batch. Duplicati a livello di *business key* (**record**) restano possibili se si riesegue l'intero esperimento senza deduplicazione: argomento ripreso nel Capitolo 4.

3.4 Delta Lake: il paradigma Data Lakehouse

Delta Lake estende Parquet con un *transaction log* che garantisce proprietà ACID su object store e filesystem [3]. Il paradigma *Lakehouse* unifica flessibilità del data lake e affidabilità del warehouse.

3.4.1 Limiti di Parquet puro

Parquet da solo non gestisce scritture concorrenti né versioning: due writer che appendono possono corrompere la directory. Per predizioni cliniche, dove ogni battito classificato deve essere persistito in modo auditabile, queste lacune sono critiche.

3.4.2 Transaction log, ACID e schema

Delta mantiene `_delta_log` con operazioni append, overwrite, merge, delete. Le scritture sono atomiche; l'*optimistic concurrency control* risolve conflitti tra writer. Per default vale **schema enforcement**: un append con colonne incompatibili fallisce. Con `mergeSchema=true` (schema evolution), nuove colonne possono essere aggiunte; utile se il payload JSON guadagna attributi, ma richiede allineamento anche sul parser `from_json` a monte.

3.4.3 Time travel, VACUUM e audit clinico

Time travel permette letture per versione (`VERSION AS OF`) o timestamp (`TIMESTAMP AS OF`), ricostruendo lo stato della tabella a un istante passato. In contesti regolati, ciò supporta audit: quale versione del log predittivo era visibile quando è stato generato un alert.

VACUUM rimuove file dati non referenziati dal log oltre una retention configurabile (default 7 giorni), bilanciando spazio disco e possibilità di rollback. Policy troppo aggressive possono impedire time travel su versioni antiche: scelta da calibrare con requisiti di conservazione clinica.

3.4.4 Integrazione con Spark

Il progetto registra le estensioni Delta nella `SparkSession`:

Listing 3.1: Configurazione Delta Lake in `SparkSession`.

```
1 .config("spark.sql.extensions",  
2   "io.delta.sql.DeltaSparkSessionExtension")  
3 .config("spark.sql.catalog.spark_catalog",  
4   "org.apache.spark.sql.delta.catalog.DeltaCatalog")
```

La stessa configurazione compare in `EcgModelTrainer` e `EcgStreamingPredictor`; solo il predictor scrive su `spark-warehouse/ecg_predictions`. Il checkpoint Spark risiede in `spark-warehouse/checkpoint`. Separare metadati di offset da dati di business è buona pratica operativa.

3.5 Sintesi: requisiti e tecnologie adottate

La Tabella 3.2 collega i requisiti dell'introduzione alle scelte tecnologiche.

Tabella 3.2: Mappatura requisiti–tecnologie.

Requisito	Tecnologia	Meccanismo
Disaccoppiamento ingestion/processing	Kafka	Topic <code>ecg-input-stream</code> , partizioni, offset
Training su dati storici	Spark MLlib	<code>EcgModelTrainer</code> , split 80/20, <code>Pipeline.fit</code>

Tabella 3.2 — *continua*

Requisito	Tecnologia	Meccanismo
Inferenza near-real-time	Structured Streaming	Micro-batch da Kafka, <code>pipelineModel.transform</code>
Persistenza affidabile	Delta Lake	Append transazionale, <code>time travel</code> , <code>foreachBatch</code>
Fault tolerance	Checkpoint Spark	<code>STREAMING_CHECKPOINT_PATH</code>
Linguaggio unificato	Scala 2.13	Batch e streaming nello stesso codebase

Il capitolo successivo traduce queste scelte in un’architettura concreta: flusso dei dati, moduli (`EcgModelTrainer`, `EcgPatientSimulator`, `EcgStreamingPredictor`) e orchestrazione tramite `AppRunner`.

Capitolo 4

Progettazione e Architettura del Sistema

4.1 Pattern architetturali: Lambda, Kappa e posizionamento del progetto

Le architetture per l'elaborazione di dati in tempo reale si sono consolidate attorno a due modelli di riferimento [11], [12], [16].

L'**architettura Lambda** prevede due pipeline parallele: un *batch layer* che calcola viste corrette su dati storici e uno *speed layer* che gestisce gli aggiornamenti recenti con latenza ridotta; un *serving layer* unifica i risultati per le query. La forza della Lambda è la correttezza batch su volumi elevati; il costo è la duplicazione della logica (stesse aggregazioni implementate due volte) e la complessità operativa.

L'**architettura Kappa** propone un unico flusso di eventi: lo speed layer consuma il log; il riprocessamento storico si ottiene rileggendo il topic dall'inizio con job batch-equivalenti. Si elimina la duplicazione logica, ma servono formati immutabili, retention adeguata e capacità di ricalcolo su scala.

Il progetto di questa tesi adotta un **ibrido pragmatico**. L'addestramento avviene in batch off-line (`EcgModelTrainer`); l'inferenza opera in streaming (`EcgStreamingPredictor`). Non esiste uno speed layer che ricalcoli aggregati su finestra mobile: il modello è un artefatto statico ricaricato dal filesystem fino a un nuovo training. La configurazione è vicina alla Lambda con confine netto training/serving, pratica comune nei sistemi ML operativi dove la frequenza di retraining è ordini di grandezza inferiore a quella di inferenza [24].

Rispetto alla Kappa pura, manca la possibilità di riaddestrare rileggendo soltanto Kafka: il training legge CSV storici, non il topic. Rispetto alla Lambda pura, manca un serving layer che fondo viste batch e real-time; le predizioni sono scritte direttamente su Delta Lake dal predictor.

4.2 Panoramica della pipeline

La pipeline si articola in tre moduli funzionali e un orchestratore:

1. **Batch layer** — `EcgModelTrainer`: legge CSV storici, addestra la pipeline ML, salva il modello su filesystem.
2. **Ingestion layer** — `EcgPatientSimulator`: simula un dispositivo edge, pubblica battiti JSON su Kafka.
3. **Streaming layer** — `EcgStreamingPredictor`: consuma Kafka, classifica, persiste su Delta Lake.
4. **Orchestratore** — `AppRunner`: coordina l'avvio sequenziale dei componenti.

La Figura ?? mostra il flusso logico dei dati; la Figura ?? ne dettaglia la sequenza temporale di avvio.

4.2.1 Flusso granulare dei dati

A livello di trasformazioni, il percorso dei dati può essere descritto come segue:

1. **CSV** → `DataFrame Spark` (`header`, `inferSchema`) nel trainer; pulizia `na.drop`, split 80/20, `Pipeline.fit`, salvataggio directory `ML_MODEL_PATH`.
2. **CSV** → subset di 1000 righe → `to_json(struct(*))` → array sul driver → `KafkaProducer.send` con chiave `beat-{i}`.
3. **Kafka** → `readStream` → cast a stringa → `from_json` con schema inferito dal CSV di training → `pipelineModel.transform`.
4. **Predizioni** → proiezione (`record`, `true_label`, `prediction`, `probability`) → `foreachBatch` → append Delta + commit checkpoint offset.

Il modello ML è l'unico artefatto condiviso tra batch e streaming (`PipelineModel.load`). Kafka funge da cuscinetto temporale; Delta materializza l'output per analisi e audit.

4.3 Vista fisica e deployment

In ambiente sperimentale, l'intera pipeline gira su un singolo host di sviluppo. La Figura ?? illustra la disposizione fisica.

4.3.1 Limiti del setup mono-nodo

`SPARK_MASTER = "local[*]"` impiega tutti i core su un'unica macchina; Kafka ha replication factor 1. Un crash del host interrompe l'intera pipeline. In produzione: cluster Spark dedicato, Kafka multi-broker con $RF \geq 3$, Delta su object store (S3, ADLS). Il setup attuale è deliberatamente minimalista per dimostrare l'integrazione end-to-end.

4.4 Fase di addestramento: `EcgModelTrainer`

Il modulo batch implementa il ciclo completo di machine learning supervisionato.

4.4.1 Flusso operativo

1. Lettura CSV da `DATASET_PATH` con header e inferenza schema.
2. Pulizia: `na.drop()` elimina righe con valori mancanti.
3. Split train/test 80/20 con `seed = 42L` per riproducibilità.
4. Costruzione pipeline: `StringIndexer` $\rightarrow$ `VectorAssembler` $\rightarrow$ `RandomForestClassifier`.
5. Addestramento (`pipeline.fit`), valutazione su test set, salvataggio su `ML_MODEL_PATH`.

4.4.2 Scelta del classificatore e alternative

Random Forest è configurato con `numTrees=100` e `maxDepth=15` [6]. La scelta risponde alla natura tabellare delle feature (intervalli pre-RR/post-RR, morfologia QRS e intervalli PQ/QT/-ST su due derivazioni) e alla capacità degli ensemble di catturare interazioni non lineari senza normalizzazione rigorosa.

Alternative valutabili nello stesso stack Spark includono:

- **GBTClassifier** (gradient boosting nativo MLlib): spesso competitivo su dati tabellari, con tempi di training superiori e maggiore rischio di overfitting senza tuning;
- **LinearSVC / LogisticRegression**: baseline interpretabili, meno adatte se le interazioni tra feature RR e morfologia QRS sono forti;
- **Reti neurali**: richiederebbero export e serving fuori dalla Pipeline MLlib, aumentando la complessità operativa senza beneficio dimostrato su questo dataset nel prototipo.

Feature importance. Random Forest fornisce, per ogni albero, una misura di impurezza ridotta per split; aggregando sui 100 alberi si ottiene un ranking delle feature utile per dialogare con il dominio clinico (quali descrittori RR o QRS guidano le confusions tra N e V). Non è implementata nel codice attuale, ma resta un'estensione naturale per migliorare l'interpretabilità senza cambiare architettura.

Cross-validation. MLlib espone `CrossValidator` e `TrainValidationSplit` per grid search su `numTrees` e `maxDepth`. Non sono stati usati: il focus della tesi è l'integrazione streaming, e il dataset preprocessato ha dimensione moderata; un tuning sistematico migliorerebbe la generalizzazione a costo di tempi di sperimentazione molto più lunghi.

4.4.3 Output del trainer

Il modello salvato include metadata, stages serializzati e statistiche dell'indicizzatore. In streaming, etichette mai viste in training sono gestite da `setHandleInvalid("keep")`, che assegna un indice dedicato anziché sollevare eccezioni: robustezza operativa a scapito di predizioni potenzialmente poco informative su classi *unknown*.

4.5 Simulazione IoT: `EcgPatientSimulator`

Il simulatore riproduce un dispositivo wearable che trasmette battiti verso un broker centralizzato.

4.5.1 Logica di produzione

1. Lettura di al massimo 1000 record dal CSV (`.limit(1000)`).
2. Serializzazione JSON tramite `to_json(struct(col("*")))`.
3. Raccolta in memoria (`.collect()`) e invio sequenziale con `KafkaProducer`.
4. Intervallo di un secondo tra messaggi (`Thread.sleep(1000)`).

L'uso di `KafkaProducer` Java puro, anziché `df.writeStream.format("kafka")`, avvicina il simulatore a un producer esterno a Spark, come avverrebbe su firmware edge senza motore distribuito.

4.5.2 Schema del payload e impatto sulla banda

Ogni messaggio serializza *tutte* le colonne del CSV: identificativo `record`, etichetta `type`, feature numeriche e metadati. La chiave `beat-{index}` serve al debug ma non è sfruttata per partizionamento semantico (paziente, sessione). In produzione, si potrebbe ridurre il payload inviando solo le feature richieste dal modello, con risparmio di banda e minor superficie di esposizione dati; il contratto andrebbe però versionato esplicitamente (Schema Registry, vedi §4.6.2).

4.5.3 Estensioni e limiti

Il rate fisso di 1 msg/s è una semplificazione rispetto a un Holter reale (jitter, burst dopo riconnessione, gap di trasmissione). Estensioni naturali: intervallo casuale gaussiano intorno alla

frequenza cardiaca; simulazione di disconnessioni; ripresa con `startingOffsets` e idempotenza lato consumer.

`.collect()` materializza 1000 record sul driver: accettabile in demo, inaccettabile per flussi continui. Questi limiti sono consapevoli e discussi nelle conclusioni.

4.6 Inferenza in tempo reale: `EcgStreamingPredictor`

Il predictor è il cuore operativo della pipeline in fase di serving.

4.6.1 Bootstrap dello schema JSON

Aniché hardcodare uno `StructType`, il codice legge lo schema dal CSV di training:

Listing 4.1: Inferenza schema da CSV per `from_json`.

```
1 val jsonSchema = spark.read
2   .option("header", "true")
3   .option("inferSchema", "true")
4   .csv(datasetPath)
5   .schema
```

Vantaggio: nessuna duplicazione manuale tra producer e consumer. Svantaggio: accoppiamento implicito al layout del CSV; una modifica delle colonne richiede riavvio coerente di simulatore e predictor.

4.6.2 Schema Registry e contratto dati

In ambienti industriali, **Confluent Schema Registry** (o equivalenti) gestisce schemi Avro, JSON Schema o Protobuf con regole di compatibilità (*backward*, *forward*). Il producer registra la versione *v* del messaggio; il consumer la risolve a runtime, fallendo in modo controllato se il contratto non è compatibile. Per il prototipo accademico, l'inferenza da CSV è sufficiente; per produzione clinica, un contratto esplicito riduce il rischio di parsing silenzioso con campi mancanti o tipi errati dopo aggiornamento firmware del dispositivo.

4.6.3 Pipeline di inferenza

1. `readStream.format("kafka")` con `startingOffsets=earliest` (solo al primo avvio senza checkpoint).
2. Cast del payload a stringa, parsing con `from_json`.
3. Trasformazione tramite `pipelineModel.transform`.
4. Proiezione su `record`, `true_label`, `prediction`, `probability`.
5. Scrittura su Delta Lake via `foreachBatch` con checkpoint.

4.6.4 Ciclo di vita del modello in produzione

Il pattern attuale — modello su directory locale, caricato all’avvio del predictor — è tipico dei prototipi ma insufficiente per operazioni mature [24]:

- **Model serving:** registry (MLflow, servizio REST) con versioni immutabili e metadati di training;
- **A/B testing:** routing di una frazione del traffico su modello candidato;
- **Model drift:** monitoraggio distribuzione feature e calo di metriche, che innesca retraining;
- **Hot-swap:** sostituzione del modello senza fermare il job streaming (richiede doppio caricamento o query restart controllato).

Nessuno di questi meccanismi è implementato; il trainer sovrascrive `ML_MODEL_PATH` e il predictor deve essere riavviato per caricare una nuova versione.

4.6.5 Fault tolerance e exactly-once

Il checkpoint in `STREAMING_CHECKPOINT_PATH` registra offset Kafka e metadati del query plan. In caso di failure del driver, il restart riprende dall’ultimo checkpoint committato. Checkpoint + append idempotente su Delta garantisce exactly-once a livello di micro-batch; duplicati di business key restano possibili se si ripete l’esperimento senza deduplicazione.

4.7 Orchestrazione: AppRunner

AppRunner coordina l’avvio in sequenza logica:

1. Verifica esistenza del modello in `ML_MODEL_PATH`; se assente, invoca `EcgModelTrainer.main`.
2. Avvia `EcgStreamingPredictor` in un thread demone (`setDaemon(true)`).
3. Attende 15 secondi (`Thread.sleep(15000)`) per il warmup di Spark Streaming.
4. Avvia `EcgPatientSimulator`.
5. Attende la terminazione del thread predictor (`predictorThread.join()`).

L’attesa di 15 secondi è un’euristica che evita che i primi messaggi arrivino prima che la query streaming sia attiva; in produzione andrebbe sostituita da health-check (`isActive`, connettività Kafka, readiness del consumer group).

4.8 Requisiti non funzionali

4.8.1 Disaccoppiamento

Kafka isola producer e consumer: il simulatore può terminare dopo 1000 messaggi mentre il predictor consuma il backlog. Il trainer batch è indipendente dal flusso streaming, eseguibile on-demand o schedulato (cron, Airflow).

4.8.2 Idempotenza e consistenza

Delta Lake append con checkpoint Spark produce un log immutabile di predizioni. Ripetere l'esperimento con gli stessi dati genera righe duplicate (non idempotenza su `record`); un MERGE Delta su chiave risolverebbe il problema.

4.8.3 Scalabilità

La scalabilità orizzontale passa dal parallelismo delle partizioni Kafka e dal numero di executor Spark. Con topic a partizione singola, il consumo resta serializzato; aumentare partizioni e parallelismo sblocca throughput superiore, come stimato nel Capitolo ??.

4.8.4 Evolvibilità dello schema

L'approccio JSON implicito è fragile: Delta può evolvere lo schema in append, ma `from_json` fallisce su tipi incompatibili o campi obbligatori mancanti. Formati schema-first (Avro, Protobuf) e Registry mitigano il rischio.

4.8.5 Observability

Il prototipo si affida a log su console (`println`, `show` nei batch) e alla Spark UI locale. In produzione servirebbero metriche esportate (throughput consumer, lag Kafka, latenza micro-batch, error rate parsing) verso Prometheus/Grafana o equivalenti cloud, con tracing distribuito per correlare `batchId` e offset. Senza observability strutturata, diagnosticare colli di bottiglia o drift operativo resta difficile oltre la demo controllata.

4.8.6 Sicurezza

L'istanza Kafka in Docker espone PLAINTEXT su localhost: nessuna autenticazione SASL, nessun TLS. I dati ECG, anche come feature, sono informazioni sanitarie sensibili: un deployment reale richiederebbe cifratura in transito (TLS tra edge, broker e Spark), controllo accessi al topic, cifratura at-rest su Delta e policy di retention allineate al GDPR. Il prototipo accademico omette questi controlli per ridurre la frizione sperimentale; vanno elencati esplicitamente come gap verso un sistema clinicamente deployabile.

Il capitolo successivo entra nel dettaglio implementativo (configurazione, listing di codice, algoritmo `foreachBatch`), mentre il Capitolo ?? presenta metriche e misure di performance.

Bibliografia

- [1] Apache Software Foundation, *KIP-500: Replace ZooKeeper with a Self-Managed Metadata Quorum*, <https://cwiki.apache.org/confluence/display/KAFKA/KIP-500>, Accessed: 2026-05-27, 2023.
- [2] M. Armbrust, T. Das, L. Sun, B. Yavuz, S. Cohen, A. Li, T. Xu, M. Zhu, A. Ghodsi, M. Zaharia, R. Xin, M. J. Franklin, I. Stoica e A. Roes, «Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark at Scale,» in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018, pp. 601–613. DOI: 10.1145/3183713.3190644.
- [3] M. Armbrust, A. Ghodsi, R. Xin e M. Zhu, «Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores,» in *Proceedings of the VLDB Endowment*, vol. 13, 2020, pp. 3411–3424. DOI: 10.14778/3415744.3415760.
- [4] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, A. Talwalkar, M. J. Franklin, A. Ghodsi, M. Zaharia e I. Stoica, «Spark SQL: Relational Data Processing in Spark,» in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 1383–1394. DOI: 10.1145/2723372.2742797.
- [5] Association for the Advancement of Medical Instrumentation, *Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms*, AAMI, 2012.
- [6] L. Breiman, «Random Forests,» *Machine Learning*, vol. 45, n. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324.
- [7] P. de Chazal, M. O'Dwyer e R. B. Reilly, «Automatic Classification of Heartbeats Using ECG Morphology and Heartbeat Interval Features,» *IEEE Transactions on Biomedical Engineering*, vol. 51, n. 7, pp. 1196–1206, 2004. DOI: 10.1109/TBME.2004.827359.
- [8] G. D. Clifford, F. Azuaje e P. McSharry, cur., *Advanced Methods and Tools for ECG Data Analysis*. Artech House, 2006, ISBN: 978-1580539667.
- [9] European Parliament and Council of the European Union, *Regulation (EU) 2017/745 on Medical Devices*, Official Journal of the European Union, L 117, 2017.
- [10] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng e H. E. Stanley, «PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals,» *Circulation*, vol. 101, n. 23, e215–e220, 2000. DOI: 10.1161/01.CIR.101.23.e215.
- [11] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017, ISBN: 978-1449373320.
- [12] J. Kreps, «Questioning the Lambda Architecture,» *O'Reilly Radar*, 2014, Online article.
- [13] J. Kreps, N. Narkhede e J. Rao, «Kafka: A Distributed Messaging System for Log Processing,» in *Proceedings of the NetDB Workshop*, 2011.

- [14] Q. Li, R. G. Mark e G. D. Clifford, «Robust Heart Rate Estimation from Multiple Asynchronous Noisy Sources Using Signal Quality Indices and a Kalman Filter,» *Physiological Measurement*, vol. 29, n. 1, pp. 15–27, 2008. DOI: 10.1088/0967-3334/29/1/002.
- [15] J. Malmivuo e R. Plonsey, *Bioelectromagnetism: Principles and Applications of Bioelectric and Biomagnetic Fields*. Oxford University Press, 1995, ISBN: 978-0195058239.
- [16] N. Marz e J. Warren, *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, 2015.
- [17] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia e A. Talwalkar, «MLlib: Machine Learning in Apache Spark,» *Journal of Machine Learning Research*, vol. 17, n. 34, pp. 1–7, 2016.
- [18] G. B. Moody e R. G. Mark, «The Impact of the MIT-BIH Arrhythmia Database,» *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, n. 3, pp. 45–50, 2001. DOI: 10.1109/51.932724.
- [19] T. Neumann, «Efficiently Compiling Efficient Query Plans for Modern Hardware,» in *Proceedings of the VLDB Endowment*, vol. 4, 2011, pp. 539–550. DOI: 10.14778/2002938.2002940.
- [20] M. Odersky, L. Spoon e B. Venners, *Programming in Scala: A Comprehensive Step-by-Step Guide*. Artima Press, 2008.
- [21] J. Pan e W. J. Tompkins, «A Real-Time QRS Detection Algorithm,» *IEEE Transactions on Biomedical Engineering*, vol. 32, n. 3, pp. 230–236, 1985. DOI: 10.1109/TBME.1985.325532.
- [22] S. Sakib, *ECG Arrhythmia Classification Dataset*, Kaggle, Versione tabellare preprocessata del MIT-BIH e di altri corpus PhysioNet, 2021. indirizzo: <https://www.kaggle.com/datasets/sadmansakib7/ecg-arrhythmia-classification-dataset>.
- [23] S. Sakib, M. Fouda e Fadlullah, Zubair Md, «Harnessing Artificial Intelligence for Secure ECG Analytics at the Edge for Cardiac Arrhythmia Classification,» in *Artificial Intelligence for Edge Computing*, Taylor & Francis, 2021. DOI: 10.1201/9781003028635-11.
- [24] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo e D. Dennison, «Hidden Technical Debt in Machine Learning Systems,» in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [25] Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology, «Heart Rate Variability: Standards of Measurement, Physiological Interpretation, and Clinical Use,» *Circulation*, vol. 93, n. 5, pp. 1043–1065, 1996. DOI: 10.1161/01.CIR.93.5.1043.
- [26] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker e I. Stoica, «Spark: Cluster Computing with Working Sets,» in *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing (HotCloud)*, 2010.
- [27] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker e I. Stoica, «Apache Spark: A Unified Engine for Big Data Processing,» *Communications of the ACM*, vol. 59, n. 11, pp. 56–65, 2016. DOI: 10.1145/2934664.